

Подготовка к экзамену по C++ (ИТМО)

Тема: Иерархия Итераторов и Требования к Контейнерам

Hardcore Theory Edition

17 июня 2026 г.

1 Категории Итераторов (Iterator Categories)

Итераторы в C++ строго выстроены в иерархию (от слабых к сильным). Сильный итератор умеет всё то же самое, что и слабый, плюс новые фишки. Алгоритмам (например, `std::sort`) плевать на контейнер, они смотрят **только на категорию итератора**.

1.1 1. Input / Output Итераторы (Самые слабые)

- **Input:** Только чтение (`*it`), только шаг вперед (`++it`). Однопроходный (нельзя вернуться назад). *Пример: чтение из `std::cin`.*
- **Output:** Только запись (`*it = 5`), только шаг вперед. *Пример: запись в файл или `std::back_inserter`.*

1.2 2. Forward Iterator (Однонаправленный)

Умеет читать и писать, идет только вперед. **Главное отличие:** многопроходный. Можно сохранить итератор в переменную, пройти вперед, а потом вернуться к сохраненному месту. *Примеры: `std::forward_list`, `std::unordered_map`, наш самописный `Range`.*

1.3 3. Bidirectional Iterator (Двунаправленный)

Умеет всё то же самое, плюс умеет **делать шаг назад** (`-it`). *Примеры: `std::list`, `std::map`, `std::set`.*

1.4 4. Random Access Iterator (Произвольный доступ)

Умеет прыгать на любое расстояние за $O(1)$. Поддерживает арифметику: `it + 5`, `it - 2`, `it1 < it2`, `it[n]`. Алгоритм `std::sort` требует именно этот тип итератора! *Примеры: `std::deque`.*

1.5 5. Contiguous Iterator (Непрерывный — C++17)

Делает всё, что Random Access, но дополнительно **гарантирует**, что элементы физически лежат в памяти друг за другом (можно кастовать к сырому указателю). *Примеры: `std::vector`, `std::array`, `std::string`, сырые указатели `T*`.*

2 Требования к Контейнерам (Named Requirements)

В стандарте C++ нет единого "базового класса" Контейнер. Вместо этого есть "Требования" (Контракты/Концепты). Если ваш класс выполняет требования — STL алгоритмы будут с ним работать.

2.1 Базовые требования

- **Container:** Должен владеть элементами, иметь `begin()`, `end()`, `size()`, `empty()` и внутренние `typedef`-ы (например, `value_type`, `iterator`). Подходят все контейнеры STL.
- **ReversibleContainer:** Должен уметь обходиться с конца. Имеет `rbegin()` и `rend()`. (Все, кроме `forward_list` и `unordered_map`).
- **AllocatorAwareContainer:** Знает, откуда брать память (принимает `std::allocator`). (Почти все контейнеры STL, кроме `std::array`, так как он живет на стеке).

2.2 Архитектурные требования

- **SequenceContainer (Последовательный):** Элементы лежат в том порядке, в котором их вставили. Никакой магии сортировки. (*Vector, Deque, List, Array*).
- **ContiguousContainer (Непрерывный):** Подкатегория последовательных. Гарантирует, что память выделена ОДНИМ сплошным куском. Адрес n -го элемента = адрес 0-го + $n \times \text{sizeof}(T)$. Обязан иметь метод `data()`. (*Vector, Array, String*).
- **AssociativeContainer (Ассоциативный):** Доступ по Ключу. Контейнер сам поддерживает строгий порядок элементов (сортировку) на основе красно-черных деревьев. Требуется `operator<`. (*Map, Set*).
- **UnorderedAssociativeContainer (Неупорядоченный):** Доступ по Ключу, но порядка нет. Хэш-таблица. Время поиска амортизированное $O(1)$. Требуется `std::hash` и `operator==`. (*Unordered_map, Unordered_set*).

Суть на пальцах (Жизненная аналогия)

Как это всё связано? Требования к контейнеру напрямую диктуют, какой итератор он сможет вам дать!

- **ContiguousContainer** (Вектор) → дает **Contiguous Iterator**.
- **SequenceContainer** типа Очередь (*Deque*) → дает **Random Access Iterator**.
- **AssociativeContainer** (Дерево) → дает **Bidirectional Iterator** (по веткам можно идти вверх и вниз, но перепрыгнуть сразу через 5 веток нельзя).